Trabalho Prático n.º 3
UC - Métodos Numéricos

202405814 Guilherme Rodrigues
202406361 Lara Dinis
202405276 Miguel Ribeiro

Dezembro 2025

Introdução

No âmbito do desenvolvimento deste trabalho, visamos aplicar os metódos de interpolação polinomial e de interpolação por splines aplicadas, quer na aproximação da função $f(x) = x^2 + \sin(6x)$ no intervalo $[-1, 1]$, quer na modelação da evolução mensal da evaporação média registada ao longo de 23 anos numa estação meteorológica em Adelaide.

Para tal, fizemos uso da linguagem de programação Python na implementação dos polinómios interpoladores de Lagrange e de Newton e do spline cúbico natural, pela sua sintaxe simples e pela facilidade com que permite implementar métodos numéricos.

Pretendemos, assim, comparar o comportamento dos métodos, analisar os erros envolvidos e justificar qual aproximação é mais adequada em cada problema.

Exercício 1

O objetivo deste exercício é estudar diferentes métodos de interpolação para aproximar a função $f(x) = x^2 + \sin(6x)$ no intervalo $[-1, 1]$.

Para o efeito, pretende-se construir um conjunto de pontos igualmente espaçados, determinar o polinómio interpolador associado e o spline cúbico natural, e comparar a qualidade das aproximações obtidas.

1. (a) Partição de [-1,1] em Subintervalos de Igual Amplitude

Para iniciar o exercício, foi necessário construir uma partição uniforme do intervalo $[-1, 1]$ contendo $n + 1 = 8$ pontos o que implica $n = 7$ subintervalos e, consequentemente, um polinómio interpolador de grau 7. Como a amplitue do intervalo é 2 (isto é, $1 - (-1) = 2$), a amplitue de cada subintervalo é dada por:

$$h = \frac{2}{7}.$$

Com recurso às bibliotecas `numpy`, utilizada para cálculos numéricos, e `sympy`, utilizada para manipulação simbólica, em Python, este processo pode ser facilmente automatizado, pelo que procedemos à definição da função e criação dos vetores de dados:

- `x_dados`, contendo as abcissas dos pontos;
- `y_dados`, contendo as imagens correspondentes.

A geração dos 8 pontos igualmente espaçados é feita de modo automático pela função `linspace`, que calcula internamente o valor do passo entre os pontos.

```
import numpy as np
import sympy as sp

def f(val):
    return val**2 + np.sin(6*val)

x_dados = np.linspace(-1, 1, 8)
y_dados = f(x_dados)
```

Listing 1: Inicialização do programa e construção dos subintervalos.

Ao executar este pedaço de código, obtemos os seguintes resultados:

i	x_i	$f(x_i) = x_i^2 + \sin(6x_i)$
0	-1.0000	1.2794
1	-0.7143	1.4206
2	-0.4286	-0.3561
3	-0.1429	-0.7356
4	0.1429	0.7764
5	0.4286	0.7234
6	0.7143	-0.4001
7	1.0000	0.7206

Tabela 1: Partição do intervalo $[-1, 1]$ em 7 subintervalos iguais e correspondentes imagens, segundo $f(x) = x^2 + \sin(6x)$.

1. (b) Construção do Polinómio Interpolador

Para construir o polinómio interpolador p, utilizaremos o Método de Lagrange, uma técnica de interpolação polinomial que permite construir o único polinómio que passa exatamente por um conjunto de pontos conhecidos.

Este baseia-se na combinação de polinómios de base $L_k(x)$, construídos de modo a assumir o valor 1 no nó x_k correspondente, e 0 em todos os restantes.

Começamos, então, por calcular os as bases $L_k(x)$:

$$L_k(x) = \frac{(x-x_0)\cdots(x-x_{k-1})(x-x_{k+1})\cdots(x-x_n)}{(x_k-x_0)\cdots(x_k-x_{k-1})(x_k-x_{k+1})\cdots(x_k-x_n)} = \prod_{\substack{i=0 \\ i\neq k}}^{n} \frac{x-x_i}{x_k-x_i}, \qquad k = 0(1)n$$

```
def calcular_termos_Lk_simbolicos(x_lista, x_simbolo):
    n = len(x_lista)
    lista_Lk = []
    for k in range(n):
        L_k = 1
        for j in range(n):
            if j != k:
                numerador = x_simbolo - x_lista[j]
                denominador = x_lista[k] - x_lista[j]
                L_k = L_k * (numerador / denominador)
        lista_Lk.append(L_k)
    return lista_Lk
```

Listing 2: Cálculo dos termos de Lagrange.

Esta função cria uma lista com todas as expressões algébricas $L_k(x)$ segundo o seguinte processo:

- Inicializa a lista vazia `lista_Lk`, onde armazenar-se-ão os polinómios de Lagrange;
- A primeira iteração do ciclo percorre cada índice k, correspondendo a cada valor x_k, de modo a construir o termo $L_k(x)$;
- Para cada valor de k, a variável `L_k` é inicializada com o valor 1, elemento neutro da multiplicação;
- O ciclo interno percorre todos os índices j, construindo o produtório $\prod_{j=0, j\neq k}^{n} \frac{x-x_j}{x_k-x_j}$;
- A condição `if j != k` garante que termos iguais são ignorados, evitando a divisão por zero e respeitando a definição de $L_k(x)$;
- Verificando-se $j| = k$:
  - `numerador = x_simbolo - x_lista[j]` corresponde ao fator $(x - x_j)$;
  - `denominador = x_lista[k] - x_lista[j]` corresponde ao fator $(x_k - x_j)$;
  - `L_k = L_k * (numerador / denominador)` atualiza o valor de $L_k(x)$, multiplicando pelo novo fator da fração.

- Após o ciclo interno terminar, o polinómio completo $L_k(x)$ é adicionado à lista;
- Finalmente, retorna a lista com todos os polinómios de base de Lagrange.

A etapa final consiste em combinar as bases $L_k(x)$ com os valores da função y_k. O polinómio interpolador é dado pela soma ponderada:

$$P(x) = \sum_{k=0}^{n} y_k \cdot L_k(x)$$

A função `resolver_lagrange_completo` integra todos estes passos: define a variável simbólica x, calcula os $L_k(x)$ e realiza o somatório. Por fim, utiliza `sp.expand` para simplificar a expressão algébrica.

```
def resolver_lagrange_completo():
    x = sp.symbols('x')
    termos_L = calcular_termos_Lk_simbolicos(x_dados, x)
    polinomio = 0
    for k in range(len(y_dados)):
        polinomio += y_dados[k] * termos_L[k]
    polinomio_final = sp.expand(polinomio)
    sp.pprint(sp.N(polinomio_final, 4))
    return polinomio_final
resolver_lagrange_completo()
```

Listing 3: Montagem do polinómio final.

Ao executar a função obtemos o polinómio interpolador p pedido:

$$p = -23.06\,x^7 - 2.665 \times 10^{-15}\,x^6 + 51.05\,x^5 + 1.776 \times 10^{-15}\,x^4 - 34.24\,x^3 + 1.0\,x^2 + 5.97\,x$$

Além da interpolação global de Lagrange, implementámos também o método do Spline Cúbico Natural, abordagem que divide o intervalo em subintervalos $[x_i, x_{i+1}]$ e define um polinómio de grau 3 para cada um deles.

O algoritmo foi desenvolvido de forma a, em vez de assumir um passo h constante, é calculado o vetor de passos $h_i = x_{i+1} - x_i$ através da função `np.diff`, permitindo a reutilização deste código, independentemente do intervalo. O processo divide-se em três etapas fundamentais:

1. **Construção do Sistema Linear**
   Para determinar os momentos M_i (as segundas derivadas nos nós), é necessário resolver o sistema linear
   $$AM = B.$$
   A matriz A é tridiagonal e construída com base na continuidade da primeira derivada. O uso de um Spline Natural impõe explicitamente que a curvatura nas extremidades seja nula, ou seja, $M_0 = 0$ e $M_n = 0$, colocando-se o valor 1 nas pontas da diagonal principal de A e 0 nos elementos correspondentes do vetor B.

```
        n = len(x_lista) - 1
        h = np.diff(x_lista)

        A = np.zeros((n + 1, n + 1))
        B = np.zeros(n + 1)

        # Condicoes de Fronteira Natural : M_0 = 0 e M_n = 0
        A[0, 0] = 1.0
        A[n, n] = 1.0

        for i in range(1, n):
            # Termo M_{i-1} (h_{i-1} / 6)
            A[i, i-1] = h[i-1] / 6

            # Termo M_i (Diagonal) -> (h_{i-1} + h_i) / 3
            A[i, i] = (h[i-1] + h[i]) / 3

            # Termo M_{i+1} (h_i / 6)
            A[i, i+1] = h[i] / 6

            # Lado Direito
            termo_dir = (y_lista[i+1] - y_lista[i]) / h[i]
            termo_esq = (y_lista[i] - y_lista[i-1]) / h[i-1]
            B[i] = termo_dir - termo_esq

```

Listing 4: Cálculo de h e montagem da matriz A e vetor B

A implementação do código resulta nas duas matrizes agora definidas:

$$A = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 \\ \frac{h_0}{6} & \frac{h_0+h_1}{3} & \frac{h_1}{6} & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & \frac{h_{n-2}}{6} & \frac{h_{n-2}+h_{n-1}}{3} & \frac{h_{n-1}}{6} \\ 0 & 0 & \cdots & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{y_2-y_1}{h_1} - \frac{y_1-y_0}{h_0} \\ \vdots \\ \frac{y_n-y_{n-1}}{h_{n-1}} - \frac{y_{n-1}-y_{n-2}}{h_{n-2}} \\ 0 \end{bmatrix}$$

Nota: A matriz A contém os coeficientes geométricos baseados nos passos h, enquanto o vetor B contém as diferenças divididas de segunda ordem (variação dos declives) calculadas pelas variáveis `termo_dir` *e* `termo_esq`*.*

2. **Cálculo dos Momentos**
   Com a matriz A e o vetor B, resolvemos o sistema linear para encontrar o vetor M. Estes valores representam a curvatura da função em cada ponto x_i.

```
        # Resolver o Sistema Linear para obter os Momentos M
        M = np.linalg.solve(A, B)

```

Listing 5: Resolução do sistema linear.

3. **Determinação dos Coeficientes**

   Conhecendo os valores de M, podemos calcular os coeficientes a_i, b_i, c_i, d_i para a equação do spline em cada intervalo i, dada por $S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$.

(a) **O coeficiente a_i:**

Se substituirmos $x = x_i$ na fórmula do polinómio, todos os termos que contêm $(x - x_i)$ anulam-se. Fica apenas:

$$S_i(x_i) = a_i$$

Como o spline tem de satisfazer a condição de interpolação, ou seja, passar pelo ponto (x_i, y_i), então $S_i(x_i)$ tem de ser igual a y_i.

Conclusão:

$$a_i = y_i$$

(No código: `a = y_lista[i]`*)*

(b) **O coeficiente c_i:**

Como $M_0, ..., M_n$ são os valores tais que $M_i = S''(x_i), \quad i = 0, 1, \ldots, n$, derivando o polinómio duas vezes em ordem a x:

$$S_i'(x) = b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2$$
$$S_i''(x) = 2c_i + 6d_i(x - x_i)$$

Calculando a segunda derivada no ponto x_i (onde $x - x_i = 0$):

$$S_i''(x_i) = 2c_i$$

Como definimos M_i exatamente como sendo a segunda derivada nesse ponto ($M_i = S_i''(x_i)$), temos:

$$M_i = 2c_i \iff c_i = \frac{M_i}{2}$$

(No código: `c = M[i] / 2`*)*

(c) **O coeficiente d_i:**

Observando a expressão da segunda derivada, $S_i''(x) = 2c_i + 6d_i(x - x_i)$, vemos que é uma função linear (uma reta). O coeficiente que multiplica o x é $6d_i$, que representa o declive dessa reta.

Sabemos, também, que a segunda derivada varia de M_i (em x_i) até M_{i+1} (em x_{i+1}). O declive desta variação é dado por:

$$\text{Declive} = \frac{\text{Final} - \text{Inicial}}{\text{Passo}} = \frac{M_{i+1} - M_i}{h_i}$$

Igualando ao coeficiente da equação ($6d_i$):

$$6d_i = \frac{M_{i+1} - M_i}{h_i} \iff d_i = \frac{M_{i+1} - M_i}{6h_i}$$

(No código: `d = (M[i+1] - M[i]) / (6 * h[i])`*)*

(d) **O coeficiente b_i:** Ao pegar na equação original do polinómio, avaliamo-la no fim do intervalo ($x = x_{i+1}$):

$$y_{i+1} = a_i + b_i(h_i) + c_i(h_i)^2 + d_i(h_i)^3$$

(Nota: $x_{i+1} - x_i = h_i$)

Substituindo a_i, c_i, d_i pelas expressões que já descobrimos:

$$y_{i+1} = y_i + b_i h_i + \frac{M_i}{2}h_i^2 + \frac{M_{i+1} - M_i}{6h_i}h_i^3$$

Em ordem a b_i temos:

$$y_{i+1} - y_i = b_i h_i + \frac{M_i h_i^2}{2} + \frac{(M_{i+1} - M_i)h_i^2}{6}$$
$$\frac{y_{i+1} - y_i}{h_i} = b_i + \frac{3M_i h_i}{6} + \frac{M_{i+1}h_i - M_i h_i}{6}$$
$$b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{6}(3M_i + M_{i+1} - M_i)$$
$$b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{6}(2M_i + M_{i+1})$$

(No código: `b = (y_lista[i+1] - y_lista[i])/h[i] - (2*h[i]*M[i] + h[i]*M[i+1])/6`*)*

A implementação dos coeficientes, como vimos, é:

```
        coeficientes = []

        for i in range(n):
            a = y_lista[i]

            # c_i = M_i / 2
            c = M[i] / 2

            # d_i = (M_{i+1} - M_i) / (6 * h_i)
            d = (M[i+1] - M[i]) / (6 * h[i])

            # b_i isolado da equacao
            b = (y_lista[i+1] - y_lista[i]) / h[i] - (2*h[i]*M[i] +
    h[i]*M[i+1]) / 6

            coeficientes.append((a, b, c, d))

```

Listing 6: Cálculo dos coeficientes finais

4. **Aplicação ao exercício**

Ao aplicar o código aos dados do exercício obtemos os seguinte valores para os coeficientes:

Tabela 2: Coeficientes do Spline Cúbico Natural obtidos para alínea 1 b).

Intervalo(n pontos)	**a**	**b**	**c**	**d**
$[0, 1]$	1.2794	2.4959	0.0000	-24.5234
$[1, 2]$	1.4206	-3.5098	-21.0200	40.3919
$[2, 3]$	-0.3561	-5.6294	13.6016	5.0848
$[3, 4]$	-0.7356	3.3882	17.9600	-39.5405
$[4, 5]$	0.7764	3.9677	-15.9318	4.8876
$[5, 6]$	0.7234	-3.9393	-11.7424	41.1806
$[6, 7]$	-0.4001	-0.5642	23.5552	-27.4811

Calculamos, então, as expressões dos polinómios segundo a função:

```
        def mostrar_equacoes_spline(coefs, x_lista):
        x = sp.symbols('x')

        for i in range(len(coefs)):
            a, b, c, d = coefs[i]
            xi = x_lista[i]
            eq = a + b*(x - xi) + c*(x - xi)**2 + d*(x - xi)**3

            eq_final = sp.expand(eq)
            print(f"S{i}(x) = {eq_final}")

```

Listing 7: Construção do Spline.

Expandindo a expressão $S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$ com os coeficientes calculados, obtemos a forma expandida da função Spline $S(x)$ para todo o domínio:

$$S(x) = \begin{cases} -24.5234x^3 - 73.5701x^2 - 71.0742x - 20.7481, & -1.00 \leq x \leq -0.71 \\ 40.3919x^3 + 65.5340x^2 + 28.2858x + 2.9091, & -0.71 \leq x \leq -0.43 \\ 5.0848x^3 + 20.1392x^2 + 8.8309x + 0.1298, & -0.43 \leq x \leq -0.14 \\ -39.5405x^3 + 1.0141x^2 + 6.0988x - 0.0003, & -0.14 \leq x \leq 0.14 \\ 4.8876x^3 - 18.0265x^2 + 8.8189x - 0.1298, & 0.14 \leq x \leq 0.43 \\ 41.1806x^3 - 64.6889x^2 + 28.8170x - 2.9867, & 0.43 \leq x \leq 0.71 \\ -27.4811x^3 + 82.4433x^2 - 76.2774x + 22.0358, & 0.71 \leq x \leq 1.00 \end{cases}$$

Análise Gráfica dos Resultados

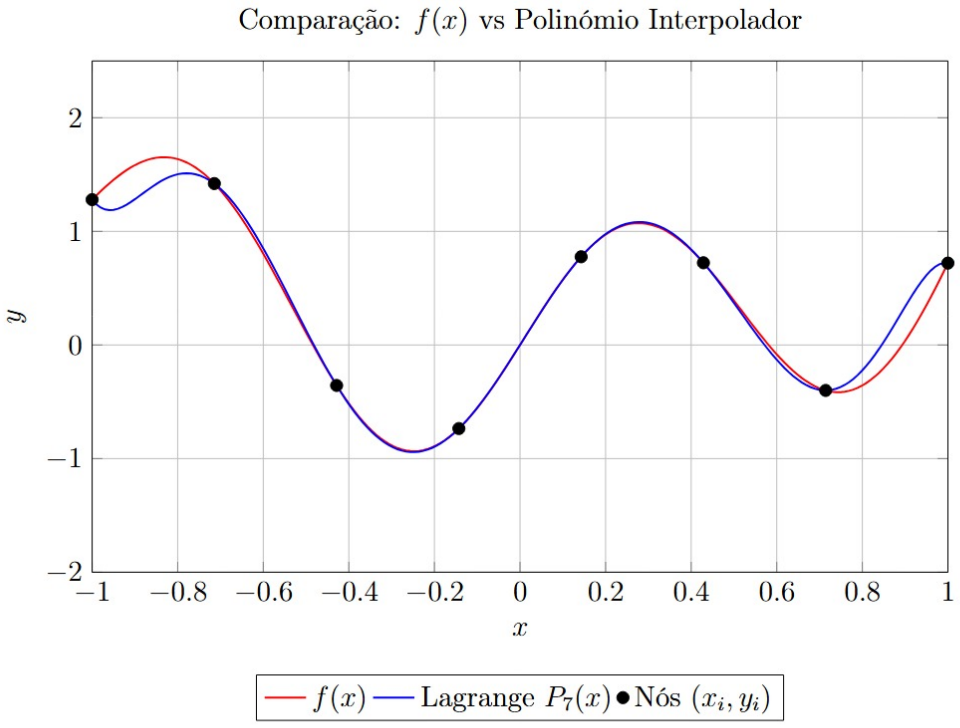


Figura 1: Comparação gráfica.

A observação da Figura 1 permite retirar as seguintes conclusões sobre o polinómio interpolador de Lagrange $P_7(x)$:

1. Tal como previsto teoricamente, o polinómio interpolador coincide exatamente com a função original nos 8 nós de interpolação assinalados, confirmando a correção no processo de construção deste.

2. No centro do intervalo, aproximadamente entre $x = -0.6$ e $x = 0.6$, observa-se uma sobreposição quase perfeita entre o polinómio e a função real: as duas curvas tornam-se praticamente indistinguíveis, indicando que nesta zona o erro de interpolação é muito reduzido.

3. À medida que nos aproximamos das extremidades do intervalo, especificamente nos subintervalos $[-1, -0.71]$ e $[0.71, 1]$, torna-se evidente uma divergência entre as curvas. O polinómio apresenta oscilações acentuadas que não refletem o comportamento suave da função original. Este comportamento é característico da interpolação polinomial com nós equidistantes e tende a agravar-se com o aumento do grau do polinómio, sendo conhecido como o Fenómeno de Runge. É precisamente para mitigar este tipo de erro que a utilização de Splines se torna uma alternativa vantajosa.

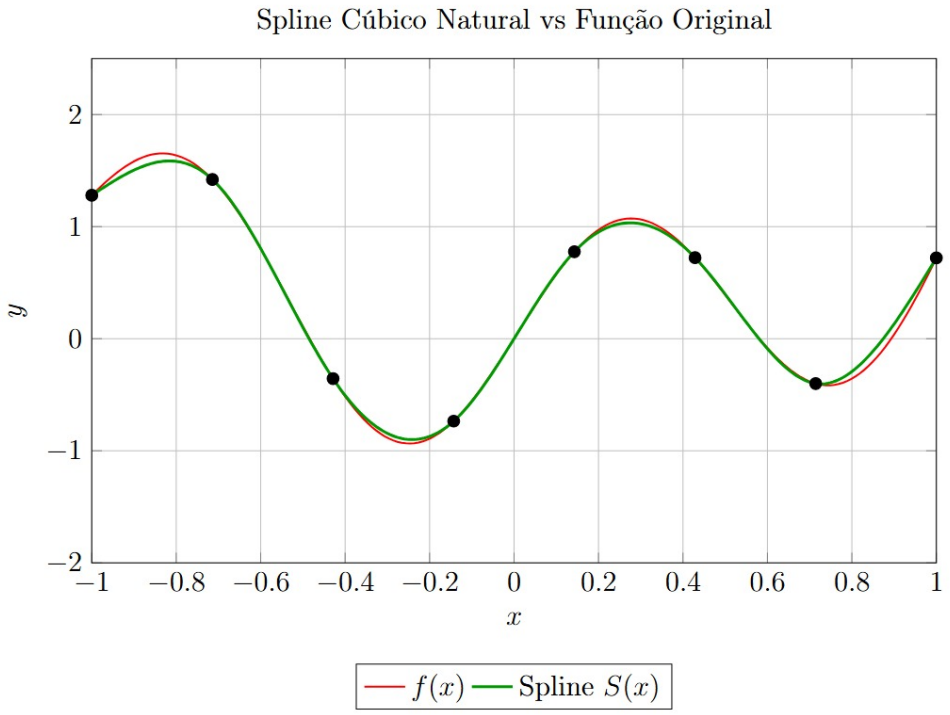


Figura 2: Representação gráfica do Spline Cúbico Natural.

1. O Spline consegue acompanhar a curvatura da função original com elevada precisão ao longo de quase todo o domínio.

2. Devido à condição do Spline Natural ($S'' = 0$ nos extremos), a curva tende a comportar-se de forma quase linear nas pontas do intervalo. Isto garante que a aproximação não "dispara"para valores irrealistas fora da zona central.

3. O contraste com o método anterior é evidente. Enquanto o polinómio de Lagrange apresentava oscilações nas extremidades, o Spline Cúbico mantém-se estável em todo o intervalo $[-1, 1]$.

   Embora o Spline possa ter um erro ligeiramente maior nos picos locais de curvatura (ex: $x \approx -0.8$), ele oferece uma aproximação global muito mais segura e controlada para este conjunto de dados.

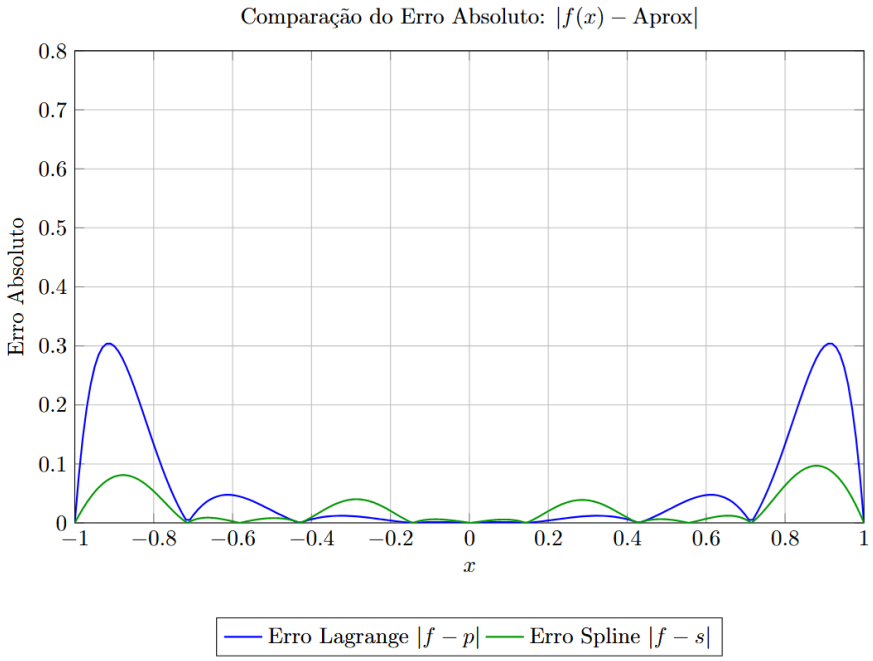


Figura 3: Erro absoluto ao longo do intervalo para os dois métodos.

A análise do gráfico confirma o resultado esperado: o erro de interpolação do Spline Cúbico é significativamente menor e mais estável do que o de Lagrange.

Enquanto o método de Lagrange sofre oscilações acentuadas nas extremidades (atingindo um erro de ≈ 0.3 devido ao fenómeno de Runge), o Spline mantém o erro controlado em todo o intervalo (máximo de ≈ 0.1), demonstrando ser a aproximação mais confiável para este conjunto de dados.

1. (c). 1. Cálculo do Majorante do Erro para o Método de Lagrange

De acordo com o Teorema do Erro na interpolação polinomial, o erro cometido ao aproximar uma função $f(x)$ por um polinómio $p_n(x)$ de grau n é dado por:

$$|f(x) - p_n(x)| \leq \frac{M_{n+1}}{(n+1)!}|\pi_{n+1}(x)| \tag{1}$$

onde:

- $n = 7$ (dado termos utilizado 8 pontos de interpolação).
- $M_{n+1} = \max_{x\in[-1,1]} |f^{(n+1)}(x)|$.
- $\pi_{n+1}(x) = \prod_{i=0}^{n}(x - x_i)$.

1. Cálculo da Derivada de Ordem Superior

A função dada é $f(x) = x^2+\sin(6x)$. Precisamos de calcular a derivada de ordem $n+1 = 8$. As derivadas sucessivas de x^2 anulam-se para ordens superiores a 2. Assim, focamo-nos no termo trigonométrico:

$$\begin{aligned} f'(x) &= 2x + 6\cos(6x) \\ f''(x) &= 2 - 36\sin(6x) \\ f'''(x) &= -216\cos(6x) \\ &\vdots \\ f^{(8)}(x) &= 6^8\sin(6x) \end{aligned}$$

Sabemos que $6^8 = 1\,679\,616$.

2. Determinação da Constante M_8

O majorante M_8 é o valor máximo da derivada no intervalo $[-1, 1]$. Como o valor máximo de $|\sin(6x)|$ é 1, temos:

$$M_8 = \max_{x\in[-1,1]} |1\,679\,616\sin(6x)| = 1\,679\,616$$

3. Cálculo do Majorante para pontos específicos

O erro é limitado por:

$$|E(x)| \leq \frac{1\,679\,616}{8!}\left|\prod_{i=0}^{7}(x - x_i)\right| = \frac{1\,679\,616}{40\,320}|\pi_8(x)| \approx 41.657 \cdot |\pi_8(x)|$$

Para ilustrar o comportamento do erro, calculamos este majorante para um ponto central ($x = 0.1$) e um ponto próximo da extremidade ($x = 0.9$):

- Para $x = 0.1$ (Centro do intervalo): O produtório das distâncias aos nós é reduzido nesta região.

$$\begin{aligned}|\pi_8(0.1)| = &|(0.1 - (-1.0000)) \cdot (0.1 - (-0.7143)) \cdot (0.1 - (-0.4286)) \\ &\cdot (0.1 - (-0.1429)) \cdot (0.1 - 0.1429) \cdot (0.1 - 0.4286) \\ &\cdot (0.1 - 0.7143) \cdot (0.1 - 1.0000)| \\ &\approx 0.000896\end{aligned}$$

$$|E_{0.1}(p)| \leq 41.657 \times 0.000896 \approx 3,73 \times 10^{-2}$$

- Para $x = 0.9$ (Extremidade do intervalo): Nas bordas, o produtório das distâncias aumenta significativamente (Fenómeno de Runge).

$$\begin{aligned}|\pi_8(0.9)| = &|(0.9 - (-1.0000)) \cdot (0.9 - (-0.7143)) \cdot (0.9 - (-0.4286)) \\ &\cdot (0.9 - (-0.1429)) \cdot (0.9 - 0.1429) \cdot (0.9 - 0.4286) \\ &\cdot (0.9 - 0.7143) \cdot (0.9 - 1.0000)| \\ &\approx 0.002817\end{aligned}$$

$$|E_{0.9}(p)| \leq 41.657 \times 0.002817 \approx 1.1173$$

4. Análise de Resultados

Os majorantes teóricos calculados para o erro da interpolação polinomial de Lagrange confirmam e quantificam o comportamento observado graficamente: no ponto central ($x = 0,1$), o majorante é, aproximadamente, $0,0373$, indicando uma boa precisão na região interior do intervalo. No entanto, na extremidade ($x = 0,9$)), o majorante sobe para cerca de $1,1173$, valor cerca de 30 vezes superior.

Esta disparidade numérica comprova que o erro do polinómio interpolador cresce drasticamente nas bordas do domínio, um efeito diretamente associado ao fenómeno de Runge, típico de interpolações polinomiais de grau elevado com nós equidistantes.

Assim, os majorantes não apenas validam as oscilações visuais nas extremidades, como também explicam a perda de fiabilidade do método de Lagrange fora da região central, reforçando a vantagem do uso de splines cúbicos em aproximações que exigem uniformidade de precisão ao longo de todo o intervalo.

1. (c). 2. Cálculo do Majorante do Erro para o Spline Cúbico

De acordo com a teoria dos splines cúbicos, para uma função $f \in C^4[a, b]$ e uma partição uniforme com amplitude máxima h, o erro de interpolação cometido pelo spline cúbico natural $S(x)$ satisfaz a seguinte majoração:

$$|f(x) - S(x)| \leq \frac{5}{384} M h^4,$$

onde

$$M = \max_{x \in [a,b]} \left| f^{(4)}(x) \right|.$$

1. Cálculo da Derivada de Ordem Superior

À semelhança do procedimento adotado na alínea anterior, iniciamos o cálculo do majorante determinando a derivada de ordem necessária, que neste caso corresponde à derivada de quarta ordem da função

$$f(x) = x^2 + \sin(6x).$$

Procedemos então ao cálculo sucessivo das derivadas:

$$\begin{aligned} f'(x) &= 2x + 6\cos(6x), \\ f''(x) &= 2 - 36\sin(6x), \\ f'''(x) &= -216\cos(6x), \\ f^{(4)}(x) &= 1296\sin(6x). \end{aligned}$$

Assim, a derivada necessária para o cálculo do majorante é

$$f^{(4)}(x) = 1296\sin(6x).$$

2. Determinação da Constante M_4

Para determinar a constante M presente na fórmula do erro, procuramos o valor máximo em módulo da derivada no intervalo de interpolação:

$$M = \max_{x\in[-1,1]} \left|f^{(4)}(x)\right| = \max_{x\in[-1,1]} |1296\sin(6x)|.$$

Como $|\sin(6x)| \leq 1$, obtemos diretamente:

$$M = 1296.$$

Este valor será posteriormente utilizado na expressão do majorante correspondente ao método do Spline Cúbico Natural, permitindo quantificar o erro máximo cometido no processo de interpolação.

3. Determinação da Constante h^4

No caso em estudo, $f(x) = x^2 + \sin(6x)$ e $[a, b] = [-1, 1]$. A partição utilizada possui $n = 7$ subintervalos de igual amplitude, pelo que

$$h = \frac{2}{7}.$$

Para a aplicação do majorante do erro do spline cúbico natural, é necessário determinar a potência h^4. Elevando o valor encontrado à quarta potência, obtemos:

$$h^4 = \left(\frac{2}{7}\right)^4 = \frac{16}{2401}.$$

Este valor será utilizado na expressão que fornece o limite superior do erro do spline cúbico.

4. Cálculo do Majorante para pontos específicos

Recordando a expressão teórica para o erro cometido pelo spline cúbico natural,

$$|f(x) - S(x)| \leq \frac{5}{384} M h^4,$$

substituindo os valores previamente determinados, podemos concluir que O erro é limitado por:

$$|f(x) - S(x)| \leq \frac{5}{384} \times 1296 \times \frac{16}{2401} \approx 1,124 \times 10^{-1}.$$

Logo,

$$|E_{0.1}(s)| = |E_{0.9}(s)| \leq 1,124 \times 10^{-1}.$$

5. Discussão

O majorante teóricos calculados para o erro do Spline Cúbico Natural é uniforme em todo o intervalo $[-1, 1]$, com um valor máximo de aproximadamente $1,124 \times 10^{-1}$.

Isto confirma a estabilidade do método, garantindo que o erro de aproximação se mantém controlado e abaixo deste limite superior em qualquer ponto do domínio.

1. (c). 3. Comparação de Resultados

Ao comparar os majorantes do erro, observa-se uma diferença significativa entre o polinômio interpolador de Lagrange e o spline cúbico.

No caso do polinômio de Lagrange, o erro é relativamente pequeno no centro do intervalo ($3,73 \times 10^{-2}$ em $x = 0,1$), mas cresce de forma acentuada nas extremidades ($1,1173$ em $x = 0,9$), evidenciando o efeito do fenômeno de Runge.

Em contraste, o spline cúbico apresenta um majorante global de $1,124 \times 10^{-1}$, mais elevado que o valor central do polinômio, mas distribuído de maneira uniforme ao longo de todo o intervalo.

Essa comparação revela que, embora o polinômio de Lagrange possa fornecer maior precisão local em pontos centrais, ele é muito sensível nas extremidades, tornando a aproximação instável quando se utiliza um conjunto equidistante de nós. O spline cúbico, por outro lado, mantém o erro relativamente baixo em todo o intervalo, garantindo uma interpolação mais equilibrada e confiável.

Assim, a análise confirma a vantagem do método de splines na obtenção de aproximações suaves e robustas, mesmo diante de nós equidistantes.

Exercício 2

2. (a) Construção do Polinómio Interpolador

Para construir o polinómio interpolador p, usaremos o método de Newton em diferenças divididas. O algorítmo que implementa o método é o seguinte:

```
    def calcular_coeficientes_diferencas_divididas(x, y):
    n = len(x)
    tabela = np.zeros((n, n))
    tabela[:, 0] = y
    for j in range(1, n):
        for i in range(n - j):
            # Fórmula: (f[i+1] - f[i]) / (x[i+j] - x[i])
            numerador = tabela[i+1][j-1] - tabela[i][j-1]
            denominador = x[i+j] - x[i]
            tabela[i][j] = numerador / denominador

    return tabela[0, :]
```

Listing 8: Cálculo dos coeficientes para diferenças divididas.

- É criada uma matriz quadrada ($n \times n$) designada por `tabela`, onde a primeira coluna é preenchida com os valores de y (as imagens $f(x_i)$), que correspondem às diferenças de ordem zero.
- Através de dois ciclos, o algoritmo preenche as colunas seguintes. Cada entrada `tabela[i][j]` é calculada com base na coluna anterior, aplicando a fórmula:

$$f[x_i, \ldots, x_{i+j}] = \frac{f[x_{i+1}, \ldots, x_{i+j}] - f[x_i, \ldots, x_{i+j-1}]}{x_{i+j} - x_i}$$

- A função retorna apenas a primeira linha da matriz (`tabela[0, :]`). Estes valores correspondem aos coeficientes $a_0, a_1, \ldots, a_n$ que serão utilizados na construção da expressão final do polinómio.

```
    def resolver_newton_simbolico():

    x = sp.symbols('x')
    coefs = calcular_coeficientes_diferencas_divididas(x_2, y_2)
    polinomio = 0
    n = len(x_2)
    for k in range(n):
        termo_atual = coefs[k]
        for i in range(k):
            termo_atual = termo_atual * (x - x_2[i])
        polinomio += termo_atual
    polinomio_final = sp.expand(polinomio)
    sp.pprint(sp.N(polinomio_final, 4))

    return polinomio_final
```

Listing 9: Cálculo do polinómio final.

Esta função é responsável pela construção do Polinómio de Newton, desenvolvendo-se do seguinte modo:

- Inicializa a variável x como um símbolo.
- Recupera os coeficientes (a_k) calculados pela função anterior e, num ciclo iterativo, constrói cada termo da forma de Newton multiplicando pelos fatores lineares acumulados:
$$a_k \cdot (x - x_0) \cdot (x - x_1) \cdots (x - x_{k-1})$$
- Soma todos os termos e utiliza o método `sp.expand` para realizar as multiplicações pendentes e simplificações, convertendo a expressão para a forma que queremos.

Obtemos, então, o seguinte polinomio:

$$\begin{aligned} P(x) = &-4.627 \times 10^{-6}x^{11} + 0.0003358x^{10} - 0.01076x^9 + 0.2004x^8 \\ &- 2.403x^7 + 19.4x^6 - 107.2x^5 + 402.4x^4 \\ &- 998.8x^3 + 1544.0x^2 - 1320.0x + 470.3 \end{aligned}$$

De modo a obter o Spline Cúbico Natural, chamamos a função do spline abordada anteriormente com o novo conjunto de dados. Ficamos então com o seguinte spline:

$$S(x) = \begin{cases} 0.4206x^3 - 1.2617x^2 - 0.7588x + 10.2000, & 1 \leq x \leq 2 \\ -1.1029x^3 + 7.8790x^2 - 19.0404x + 22.3877, & 2 \leq x \leq 3 \\ 1.1910x^3 - 12.7656x^2 + 42.8935x - 39.5462, & 3 \leq x \leq 4 \\ -0.6610x^3 + 9.4575x^2 - 45.9990x + 78.9771, & 4 \leq x \leq 5 \\ 0.4529x^3 - 7.2503x^2 + 37.5404x - 60.2552, & 5 \leq x \leq 6 \\ -0.3506x^3 + 7.2128x^2 - 49.2384x + 113.3024, & 6 \leq x \leq 7 \\ 0.2496x^3 - 5.3908x^2 + 38.9867x - 92.5561, & 7 \leq x \leq 8 \\ -0.1476x^3 + 4.1414x^2 - 37.2709x + 110.7974, & 8 \leq x \leq 9 \\ -0.0591x^3 + 1.7511x^2 - 15.7583x + 46.2598, & 9 \leq x \leq 10 \\ -0.0160x^3 + 0.4594x^2 - 2.8407x + 3.2009, & 10 \leq x \leq 11 \\ 0.0232x^3 - 0.8354x^2 + 11.4017x - 49.0213, & 11 \leq x \leq 12 \end{cases}$$

Graficamente obtemos:

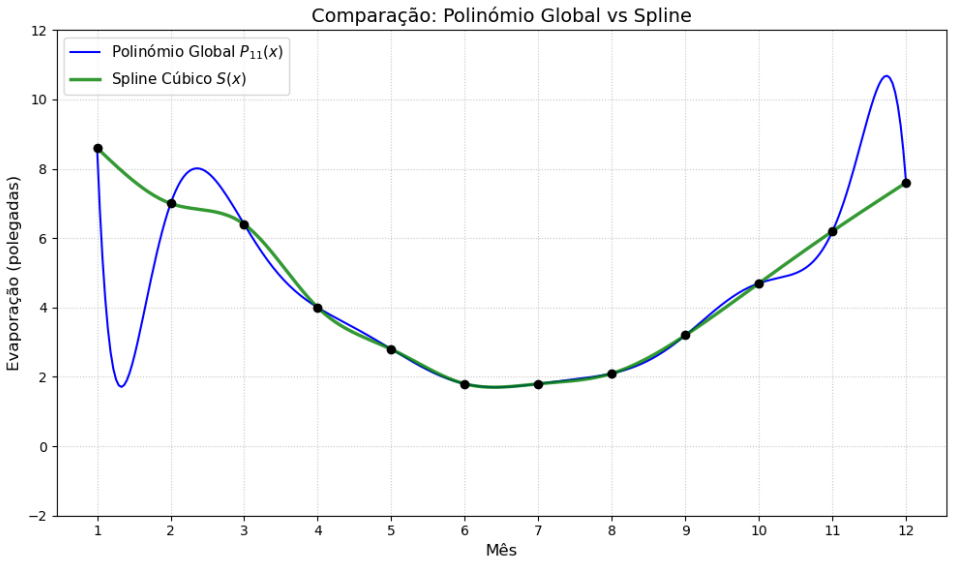


Figura 4: Gráfico comparativo entre os dois métodos.

2. (b) Análise de Resultados

Analisando o gráfico obtido e considerando o contexto do problema, concluímos que a aproximação mais adequada é a do Spline Cúbico Natural.

Esta escolha justifica-se pelo enunciado, que indica que a evaporação diminui gradualmente de janeiro a junho-julho e aumenta novamente de agosto a dezembro. Como se trata de médias climáticas calculadas ao longo de 23 anos, espera-se uma evolução suave dos valores.

Observando a Figura 4, verificamos que o polinómio obtido pelo método de Newton apresenta oscilações acentuadas e irrealistas nas extremidades, evidenciando mais uma vez o Fenómeno de Runge.

Em contraste, o spline, construído a partir de polinómios de baixo grau, em cada subintervalo, elimina essas oscilações e oferece uma representação que reflete de forma mais fiel a variação gradual observada no problema.